



Universidade Estadual Paulista "Júlio de Mesquita Filho"
Instituto de Ciência e Tecnologia de Sorocaba
Engenharia de Controle e Automação

Leticia Eguchi Andrade e Silva
Prof. Leopoldo André Dutra Lusquino Filho

RELATÓRIO DE ESTRUTURA DE DADOS: SIMULADOR DE GESTÃO ACADÊMICA

Sorocaba
2026

RESUMO

O presente trabalho apresenta o desenvolvimento de um sistema em C/C++ para armazenamento, organização e análise de dados provenientes de sensores de movimento. O sistema utiliza como base o dataset UCI HAR (Human Activity Recognition), composto por informações coletadas por acelerômetros e giroscópios de smartphones durante a execução de diferentes atividades humanas.

Foram implementadas cinco estruturas de dados para comparar seu desempenho em diferentes operações. Entre elas estão três estruturas estudadas na disciplina (Lista Encadeada, Árvore AVL e Tabela Hash) e duas estruturas adicionais (Skip List e Árvore Trie). O sistema realiza operações de inserção, busca e remoção de dados, além de executar análises estatísticas para detectar leituras consideradas anômalas.

Para tornar os testes mais representativos, o conjunto de dados foi ampliado para mais de 10.000 registros utilizando técnicas de geração de dados, como inserção de ruído e criação de novas amostras. Também foi desenvolvida uma versão otimizada da Árvore AVL utilizando bitfields, com o objetivo de reduzir o consumo de memória.

Além dos testes em condições normais, foram realizados benchmarks sob diferentes restrições de memória, processamento, latência, qualidade dos dados e limitações estruturais, permitindo avaliar o comportamento de cada estrutura em cenários semelhantes aos encontrados em sistemas embarcados.

Os resultados mostram que cada estrutura apresenta vantagens específicas, dependendo da aplicação e do tipo de operação realizada. Dessa forma, a escolha da estrutura de dados deve considerar as características do problema e os requisitos de desempenho do sistema.

Palavras-chave: Estruturas de Dados; Benchmark; Árvore AVL; Tabela Hash; Skip List; Detecção de Anomalias.

ABSTRACT

This work presents the development of a C/C++ system for storing, organizing, and analyzing data collected from motion sensors. The system is based on the UCI HAR (Human Activity Recognition) dataset, which contains information gathered from smartphone accelerometers and gyroscopes during different human activities.

Five data structures were implemented to compare their performance under different operations. Three of them are traditional data structures studied in the course (Linked List, AVL Tree, and Hash Table), while two additional structures (Skip List and Trie Tree) were also implemented.

The system performs insertion, search, and deletion operations, in addition to statistical analysis for anomaly detection. To obtain a larger dataset, data augmentation techniques such as noise insertion and synthetic sample generation were applied, resulting in more than 10,000 records.

An optimized AVL Tree using bitfields was also developed to reduce memory consumption. Furthermore, benchmark tests were carried out under different restrictive scenarios involving memory usage, processing power, latency, data quality, and algorithmic limitations.

The results demonstrate that each data structure performs better in specific situations, reinforcing that the choice of a data structure should always consider the application's requirements and the characteristics of the processed data.

Keywords: Data Structures; AVL Tree; Hash Table; Skip List; Benchmark; Anomaly Detection.

SUMÁRIO

1. INTRODUÇÃO	4
2. OBJETIVOS	5
2.1 Objetivo Geral	5
2.2 Objetivos Específicos	5
3. FUNDAMENTAÇÃO TEÓRICA	6
3.1 Lista Encadeada	6
3.2 Árvore AVL	6
3.3 Tabela Hash	7
3.4 Skip List	7
3.5 Árvore Trie	7
Considerações da Fundamentação:	8
4. METODOLOGIA, ARQUITETURA E TRATAMENTO DE DADOS	9
4.1 Dataset utilizado	9
4.2 Geração e Tratamento dos dados	9
4.3 Arquitetura do Sistema	9
4.4 Estrutura otimizada	10
4.5 Módulo de Detecção de Anomalias	10
5. BENCHMARKING E AVALIAÇÃO SOB CONDIÇÕES RESTRITIVAS	12
Restrição de Memória (R1)	12
Restrição de Processamento (R6)	12
Restrição de Latência (R11)	12
Restrição de Dados (R16)	12
Restrição Algorítmica (R22)	13
6. RESULTADOS E DISCUSSÕES	14
Tabela 1 – Desempenho médio das estruturas com aproximadamente 15.000 registros	14
6.1 Desempenho em Condições Normais	15
6.2 Comparação entre a AVL Tradicional e a AVL Otimizada	15
6.3 Comportamento da Skip List	16
6.4 Impacto das Restrições	16
6.5 Comparação com a Complexidade Teórica	17
Considerações sobre os Resultados:	17
7. CONCLUSÃO	18
8. REFERÊNCIAS BIBLIOGRÁFICAS	20

1. INTRODUÇÃO

O uso de grandes volumes de dados é cada vez mais comum em sistemas computacionais modernos. Aplicações como Internet das Coisas (IoT), monitoramento industrial, dispositivos embarcados, sistemas de saúde e cidades inteligentes dependem do processamento contínuo de informações geradas por sensores e outros dispositivos. Nesses ambientes, novos dados são inseridos constantemente, enquanto outros precisam ser consultados, atualizados ou removidos em tempo real.

Para que esse processamento seja eficiente, é fundamental escolher estruturas de dados adequadas. Cada estrutura possui características próprias que influenciam diretamente o desempenho das operações de inserção, busca, remoção e armazenamento. Enquanto uma Tabela Hash oferece consultas muito rápidas por meio de chaves, estruturas como Árvores AVL mantêm os dados ordenados e garantem um bom desempenho mesmo com grandes volumes de informação. Já estruturas como Skip List e Trie apresentam soluções alternativas para problemas específicos, podendo oferecer vantagens dependendo do tipo de aplicação.

Além da análise teórica de complexidade dos algoritmos, é importante avaliar como essas estruturas se comportam na prática. Aspectos como organização da memória, quantidade de dados, limitações de hardware e características do sistema operacional podem influenciar significativamente o desempenho observado durante a execução.

Neste projeto foi desenvolvido um sistema para armazenar e analisar dados de sensores de movimento utilizando diferentes estruturas de dados. O sistema permite inserir, remover e buscar registros, além de realizar análises estatísticas para identificar leituras consideradas anormais. Também foram realizados testes de desempenho em diferentes condições restritivas, permitindo comparar o comportamento de cada estrutura em cenários próximos aos encontrados em aplicações reais.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um sistema em linguagem C/C++ para armazenamento e processamento de dados de sensores utilizando diferentes estruturas de dados, analisando seu desempenho por meio de benchmarks realizados em condições normais e em cenários com restrições de execução.

2.2 Objetivos Específicos

Para alcançar esse objetivo, foram definidos os seguintes objetivos específicos:

- Implementar três estruturas de dados estudadas na disciplina: Lista Encadeada, Árvore AVL e Tabela Hash;
- Implementar duas estruturas adicionais: Skip List e Árvore Trie;
- Utilizar o dataset UCI HAR como base para os experimentos, ampliando sua quantidade de registros por meio de técnicas de geração de dados;
- Desenvolver um módulo de detecção de anomalias baseado em média e desvio padrão, utilizando a regra dos três sigmas para identificar leituras fora do comportamento esperado;
- Implementar uma versão otimizada da Árvore AVL utilizando bitfields para reduzir o consumo de memória;
- Realizar testes comparativos entre as estruturas implementadas, avaliando tempo de inserção, busca, remoção, consumo de memória e comportamento sob diferentes condições restritivas;
- Comparar os resultados obtidos experimentalmente com a complexidade teórica de cada estrutura, analisando suas vantagens e limitações para diferentes cenários de utilização.

3. FUNDAMENTAÇÃO TEÓRICA

As estruturas de dados são responsáveis por organizar as informações de forma que operações como inserção, busca, remoção e atualização sejam realizadas com eficiência. A escolha da estrutura mais adequada depende das necessidades da aplicação, pois cada uma apresenta vantagens e limitações em relação ao desempenho, ao consumo de memória e à forma de armazenamento dos dados.

Neste projeto foram utilizadas cinco estruturas de dados diferentes. Três delas fazem parte do conteúdo da disciplina (Lista Encadeada, Árvore AVL e Tabela Hash), enquanto as outras duas (Skip List e Árvore Trie) foram escolhidas para ampliar a análise comparativa do sistema.

3.1 Lista Encadeada

A Lista Encadeada é uma estrutura linear formada por nós, onde cada nó armazena um conjunto de dados e um ponteiro para o próximo elemento da sequência. Diferentemente dos vetores, seus elementos não precisam estar armazenados em posições consecutivas da memória.

Sua principal vantagem está na facilidade para inserir e remover elementos, principalmente no início da lista, sem necessidade de deslocar outros registros. Entretanto, como os elementos não possuem acesso direto por posição, qualquer operação de busca exige percorrer a lista sequencialmente até encontrar o elemento desejado.

Por esse motivo, sua complexidade para inserção no início é $O(1)$, enquanto operações de busca apresentam complexidade $O(n)$.

Apesar dessa limitação, listas encadeadas continuam sendo bastante utilizadas em aplicações que realizam muitas inserções e remoções dinâmicas.

3.2 Árvore AVL

A Árvore AVL é uma árvore binária de busca auto-balanceada, criada para evitar que a árvore se torne excessivamente inclinada após diversas inserções ou remoções.

Seu funcionamento baseia-se no fator de balanceamento de cada nó, calculado pela diferença entre as alturas das subárvores esquerda e direita. Sempre que essa diferença ultrapassa os limites permitidos (-1, 0 ou 1), a árvore realiza rotações automáticas para restaurar o equilíbrio.

Esse processo garante que a altura da árvore permaneça aproximadamente proporcional a $\log_2(n)$, permitindo que operações de busca, inserção e remoção sejam executadas com complexidade $O(\log n)$.

Embora exija um pequeno custo adicional devido às rotações de balanceamento, a AVL apresenta excelente desempenho quando os dados precisam permanecer ordenados durante toda a execução do sistema.

No projeto, essa estrutura foi utilizada para comparar o desempenho das buscas ordenadas e também serviu como base para a implementação da versão otimizada utilizando bitfields.

3.3 Tabela Hash

A Tabela Hash é uma estrutura que permite localizar informações rapidamente por meio de uma chave de identificação.

Seu funcionamento consiste em aplicar uma função hash sobre a chave, transformando-a em um índice do vetor onde o elemento será armazenado. Dessa forma, o acesso aos dados ocorre praticamente de maneira direta, sem necessidade de percorrer todos os registros.

Quando duas chaves produzem o mesmo índice ocorre uma colisão. Para tratar esse problema, foi utilizado o método de encadeamento separado, no qual cada posição da tabela mantém uma lista contendo todos os elementos associados àquele índice.

Em condições ideais, operações de inserção, busca e remoção possuem complexidade média $O(1)$. Entretanto, quando há muitas colisões, o desempenho pode diminuir, aproximando-se do comportamento de uma lista encadeada.

Por sua rapidez nas consultas, a Tabela Hash é bastante utilizada em sistemas que precisam localizar informações por identificadores únicos.

3.4 Skip List

A Skip List é uma estrutura probabilística criada como alternativa às árvores balanceadas.

Ela é formada por várias listas encadeadas organizadas em diferentes níveis. A camada inferior contém todos os elementos, enquanto as camadas superiores

armazenam apenas parte deles, funcionando como atalhos que aceleram as buscas.

Sempre que um novo elemento é inserido, uma decisão probabilística determina quantos níveis esse elemento ocupará. Essa característica elimina a necessidade de rotações ou rebalanceamentos, tornando a implementação mais simples que a de árvores balanceadas.

Em média, as operações de busca, inserção e remoção apresentam complexidade $O(\log n)$, embora, em situações extremas, possam atingir $O(n)$.

Neste projeto, a Skip List foi escolhida para comparar seu desempenho com o da Árvore AVL, permitindo avaliar as diferenças entre uma estrutura probabilística e uma estrutura rigidamente balanceada.

3.5 Árvore Trie

A Trie, também conhecida como Árvore Prefixada, é uma estrutura especializada no armazenamento de cadeias de caracteres.

Ao contrário das árvores binárias tradicionais, cada nó representa apenas um caractere da palavra armazenada. Dessa forma, palavras com prefixos em comum compartilham parte da estrutura, reduzindo redundâncias e tornando as buscas muito eficientes.

Sua principal vantagem é que o tempo de busca depende apenas do tamanho da palavra pesquisada e não da quantidade de elementos armazenados na estrutura.

Por esse motivo, a Trie é bastante utilizada em sistemas de busca automática, dicionários, autocompletar, roteamento de URLs e aplicações que trabalham intensivamente com textos.

No sistema desenvolvido, essa estrutura foi utilizada para armazenar informações textuais relacionadas aos registros dos sensores, permitindo comparar seu comportamento com as demais estruturas implementadas.

Considerações da Fundamentação:

As cinco estruturas escolhidas apresentam características bastante diferentes, o que torna a comparação interessante dentro do contexto do projeto.

Enquanto a Lista Encadeada oferece simplicidade de implementação, a Árvore AVL garante que os dados permaneçam ordenados com bom desempenho. A Tabela Hash prioriza rapidez nas consultas por chave, a Skip List apresenta uma alternativa probabilística às árvores balanceadas e a Trie é especializada em buscas envolvendo cadeias de caracteres.

A análise prática dessas estruturas, realizada por meio dos benchmarks apresentados nos capítulos seguintes, permite verificar como suas propriedades teóricas se refletem em situações reais de uso.

4. METODOLOGIA, ARQUITETURA E TRATAMENTO DE DADOS

4.1 Dataset utilizado

- Para o desenvolvimento deste projeto foi utilizado o dataset UCI HAR (Human Activity Recognition Using Smartphones), um conjunto de dados amplamente utilizado em pesquisas relacionadas ao reconhecimento de atividades humanas. Esse dataset contém informações obtidas por sensores de acelerômetro e giroscópio presentes em smartphones durante a realização de atividades como caminhar, subir escadas, sentar e correr.
- A escolha desse conjunto de dados ocorreu porque ele atende aos requisitos do projeto, apresentando uma quantidade significativa de registros e informações estruturadas que podem ser armazenadas e manipuladas pelas diferentes estruturas de dados implementadas.
- Embora o dataset original possua aproximadamente dez mil amostras, foi necessário ampliar sua quantidade de registros para tornar os testes mais representativos e permitir uma melhor análise do comportamento das estruturas em diferentes cenários de execução.

4.2 Geração e Tratamento dos dados

Para aumentar a quantidade de dados disponíveis e simular um ambiente mais próximo de uma aplicação real, foi desenvolvido um módulo responsável pela geração automática de novas amostras.

Esse processo foi baseado em técnicas de Data Augmentation, permitindo expandir o conjunto de dados sem alterar significativamente suas características estatísticas.

Entre as técnicas utilizadas destacam-se:

- Inserção de ruído gaussiano nas leituras dos sensores para simular pequenas variações normalmente encontradas em dispositivos físicos;

- Alteração aleatória de identificadores e timestamps, simulando diferentes momentos de coleta;
- Geração de novos registros a partir das informações existentes, preservando a distribuição original dos dados.

Após esse processo, o sistema passou a trabalhar com um volume superior a 15.000 registros, possibilitando avaliar com maior precisão o desempenho das estruturas implementadas.

Antes do armazenamento, os dados também passaram por uma etapa de validação para verificar possíveis inconsistências e garantir que todas as estruturas recebessem exatamente o mesmo conjunto de informações durante os testes comparativos.

4.3 Arquitetura do Sistema

O sistema foi desenvolvido em linguagem C++, utilizando uma arquitetura modular, na qual cada estrutura de dados foi implementada em classes independentes.

Essa organização facilita a manutenção do código e permite que todas as estruturas executem as mesmas operações básicas, tornando a comparação entre elas mais justa.

Durante a execução, o sistema realiza continuamente as seguintes operações:

- leitura dos registros do dataset;
- inserção dos dados nas estruturas;
- busca por registros específicos;
- remoção de elementos quando necessário;
- execução das análises estatísticas;
- coleta das métricas utilizadas nos benchmarks.

Como todas as estruturas recebem exatamente o mesmo conjunto de dados e executam as mesmas operações, torna-se possível comparar seu desempenho em condições equivalentes.

4.4 Estrutura otimizada

Além das implementações convencionais, foi desenvolvida uma versão otimizada da Árvore AVL com o objetivo de reduzir o consumo de memória.

A otimização foi realizada utilizando bitfields, recurso disponível na linguagem C++, que permite armazenar informações pequenas utilizando apenas alguns bits, em vez de ocupar um inteiro completo.

Na implementação tradicional, informações como o fator de balanceamento ocupam normalmente quatro bytes. Como esse valor pode assumir apenas três possibilidades (-1, 0 e 1), é possível armazená-lo utilizando apenas alguns bits.

Essa modificação reduz o tamanho de cada nó da árvore, permitindo que uma quantidade maior de elementos seja armazenada utilizando a mesma quantidade de memória.

Embora essa otimização não altere a complexidade dos algoritmos, ela melhora o aproveitamento da memória, característica importante principalmente em sistemas embarcados e dispositivos com recursos limitados.

Durante os benchmarks, a versão otimizada foi comparada diretamente com a implementação convencional da Árvore AVL para avaliar os ganhos obtidos.

4.5 Módulo de Detecção de Anomalias

Além das operações de inserção, busca e remoção exigidas pelo projeto, o sistema também realiza uma análise estatística dos dados armazenados.

Essa funcionalidade foi implementada por meio da classe `AnomalyDetector`, responsável por identificar leituras consideradas fora do comportamento esperado.

Inicialmente, o sistema calcula a média e o desvio padrão das amostras armazenadas.

Em seguida, sempre que uma nova leitura é recebida, seu valor é comparado com esses parâmetros utilizando a regra dos três sigmas (3σ).

Quando a diferença entre a leitura e a média ultrapassa três vezes o valor do desvio padrão, o registro é classificado como uma anomalia.

Nessa situação, o sistema emite um alerta no console e remove automaticamente esse registro da estrutura de dados utilizada na simulação.

Esse procedimento permite representar uma situação semelhante à encontrada em sistemas reais de monitoramento, nos quais sensores podem apresentar leituras incorretas devido a falhas de hardware, interferências ou condições anormais de funcionamento.

5. BENCHMARKING E AVALIAÇÃO SOB CONDIÇÕES RESTRITIVAS

Após a implementação do sistema, foram realizados diversos testes com o objetivo de comparar o desempenho das estruturas de dados em diferentes situações de uso.

Os benchmarks foram desenvolvidos utilizando a biblioteca `<chrono>` da linguagem C++, permitindo medir o tempo necessário para executar operações como inserção, busca e remoção.

Também foram coletadas informações relacionadas ao consumo de memória, possibilitando analisar os trade-offs entre desempenho e utilização de recursos computacionais.

Para tornar os experimentos mais próximos de situações reais, foram simuladas cinco condições restritivas previstas no enunciado do projeto.

Restrição de Memória (R1)

Foi estabelecido um limite artificial para a memória disponível durante a execução.

O objetivo foi verificar quais estruturas conseguem armazenar uma quantidade maior de registros utilizando menos memória e como elas se comportam quando esse recurso se torna limitado.

Essa condição foi especialmente importante para avaliar a eficiência da versão otimizada da Árvore AVL.

Restrição de Processamento (R6)

Todos os testes foram executados considerando apenas um núcleo de processamento.

Essa configuração elimina vantagens relacionadas ao paralelismo e permite observar exclusivamente a eficiência dos algoritmos implementados.

Dessa forma, foi possível avaliar o impacto das rotações da Árvore AVL, das operações realizadas pela Skip List e das consultas executadas pela Tabela Hash.

Restrição de Latência (R11)

Foi introduzido um atraso artificial durante determinadas operações para simular situações em que o sistema depende de comunicação com dispositivos remotos ou bancos de dados.

Essa condição permite avaliar como o aumento da latência influencia o tempo de resposta das estruturas de dados.

Restrição de Dados (R16)

Durante essa etapa foram inseridos registros contendo ruídos e valores inconsistentes.

O objetivo foi verificar como o sistema responde quando recebe informações incorretas e como o módulo de detecção de anomalias atua para identificar e remover esses registros automaticamente.

Restrição Algorítmica (R22)

Na Skip List foi imposto um limite para a quantidade máxima de níveis disponíveis.

Essa modificação reduz sua eficiência, aproximando seu comportamento ao de uma lista encadeada tradicional.

Esse teste permitiu observar como a redução da complexidade estrutural afeta diretamente o tempo de busca e de inserção dos elementos.

Ao final de cada experimento foram registradas as métricas de tempo, consumo de memória e comportamento das estruturas, permitindo comparar seus resultados com as características teóricas estudadas ao longo da disciplina.

6. RESULTADOS E DISCUSSÕES

Após a implementação de todas as estruturas e a execução dos benchmarks, foi possível comparar o comportamento de cada uma delas em diferentes cenários de utilização. Os testes permitiram observar não apenas o tempo necessário para executar cada operação, mas também o consumo de memória e o impacto das restrições impostas durante a simulação.

Os resultados obtidos confirmam, de maneira geral, o comportamento esperado pela análise teórica das estruturas de dados. Entretanto, algumas diferenças observadas mostram que fatores como organização da memória, quantidade de dados e características da implementação também influenciam significativamente o desempenho na prática.

A Tabela 1 apresenta um resumo das principais métricas obtidas durante os testes.

Tabela 1 – Desempenho médio das estruturas com aproximadamente 15.000 registros

Métrica (unidade)	Lista Encadeada	Árvore AVL	Tabela Hash	Skip List
Inserção em Lote (ms)	0,711	5,794	0,748	3,415
Busca de ID Aleatório (μ s/OP)	27,3047	0,2123	0,1847	0,2636

Remoção Condicional (ms)	25,055	0,464	0,248	0,328
Taxa de Memória RAM (KB)	546	703	554	1796

Observando os resultados, percebe-se que nenhuma estrutura apresentou o melhor desempenho em todas as operações. Cada uma mostrou vantagens específicas de acordo com o tipo de processamento realizado, reforçando que a escolha da estrutura depende diretamente das necessidades da aplicação.

6.1 Desempenho em Condições Normais

Nos testes realizados sem restrições artificiais, a Tabela Hash apresentou o melhor desempenho nas operações de busca.

Esse resultado era esperado, já que esse tipo de estrutura realiza o acesso aos elementos por meio de uma função hash, permitindo localizar registros praticamente em tempo constante quando a distribuição das chaves é adequada.

A Árvore AVL também apresentou excelente desempenho nas buscas, mantendo tempos muito próximos aos da Hash. Entretanto, por precisar manter o balanceamento da árvore durante inserções e remoções, seu tempo de inserção foi maior.

A Skip List apresentou resultados semelhantes aos da AVL. Apesar de utilizar uma estratégia completamente diferente para organizar os dados, sua complexidade média também é logarítmica, o que explica o desempenho observado durante os testes.

Já a Lista Encadeada apresentou os maiores tempos de busca e remoção.

Esse comportamento também era esperado, pois essas operações exigem percorrer a lista sequencialmente até localizar o elemento desejado. À medida que o número de registros aumenta, essa característica faz com que o tempo de execução cresça significativamente.

Esses resultados confirmam que a análise prática está de acordo com a complexidade teórica estudada durante a disciplina.

6.2 Comparação entre a AVL Tradicional e a AVL Otimizada

Um dos objetivos do projeto foi desenvolver uma versão otimizada de uma das estruturas implementadas.

Para isso, foi escolhida a Árvore AVL, cuja estrutura foi modificada utilizando bitfields para reduzir o espaço ocupado pelos metadados de cada nó.

Os testes mostraram que essa otimização reduziu o consumo de memória da estrutura, permitindo armazenar uma quantidade maior de elementos antes de atingir os limites impostos durante os experimentos.

Em relação ao tempo de execução, praticamente não foram observadas diferenças significativas.

Esse resultado era esperado, pois a otimização alterou apenas a representação dos dados em memória, sem modificar os algoritmos responsáveis pelas operações de inserção, busca e remoção.

Mesmo sem ganhos expressivos de velocidade, a redução do consumo de memória torna essa implementação interessante para aplicações embarcadas, onde esse recurso costuma ser limitado.

6.3 Comportamento da Skip List

Durante os testes realizados em condições normais, a Skip List apresentou desempenho bastante próximo ao da Árvore AVL.

Embora utilize uma abordagem probabilística para organizar os elementos, sua eficiência média permaneceu elevada durante praticamente toda a execução.

A principal diferença apareceu durante as operações de remoção.

Enquanto a Árvore AVL precisa realizar rotações para manter seu balanceamento após determinadas exclusões, a Skip List realiza apenas ajustes nos ponteiros existentes entre seus níveis.

Como consequência, em alguns cenários envolvendo muitas remoções consecutivas, a Skip List apresentou tempos ligeiramente menores que os da AVL.

Esse comportamento demonstra que estruturas probabilísticas podem representar uma alternativa interessante quando se deseja reduzir a complexidade da implementação sem comprometer significativamente o desempenho.

6.4 Impacto das Restrições

A execução dos benchmarks sob diferentes condições restritivas permitiu observar como cada estrutura reage quando os recursos disponíveis deixam de ser ideais.

Na restrição de memória, a versão otimizada da Árvore AVL apresentou melhor aproveitamento do espaço disponível, armazenando uma quantidade maior de registros antes de atingir o limite imposto.

Na restrição de processamento, executada utilizando apenas um núcleo da CPU, as diferenças entre as estruturas ficaram mais evidentes. Operações que exigem maior

quantidade de processamento, como as rotações da AVL, passaram a consumir mais tempo quando comparadas à Tabela Hash.

Durante os testes com dados corrompidos, o módulo de detecção de anomalias conseguiu identificar corretamente registros fora do padrão utilizando a regra dos três sigmas.

Após a identificação dessas leituras, o sistema removeu automaticamente os registros considerados inválidos, mantendo apenas informações consistentes nas estruturas.

Na restrição algorítmica aplicada à Skip List, a limitação do número de níveis reduziu sua eficiência nas operações de busca. Como consequência, seu comportamento aproximou-se gradativamente do observado em uma Lista Encadeada, evidenciando a importância dos níveis superiores para manter sua complexidade média em $O(\log n)$.

Esses experimentos mostram que pequenas alterações na estrutura ou nas condições de execução podem produzir diferenças significativas no desempenho observado.

6.5 Comparação com a Complexidade Teórica

De maneira geral, os resultados experimentais foram compatíveis com as previsões teóricas estudadas durante a disciplina.

A Tabela Hash apresentou o menor tempo médio de busca, refletindo sua complexidade média $O(1)$.

A Árvore AVL e a Skip List mantiveram comportamento próximo ao esperado para estruturas logarítmicas, apresentando bom desempenho mesmo com o aumento da quantidade de registros.

Por outro lado, a Lista Encadeada apresentou crescimento praticamente linear no tempo das buscas, confirmando sua complexidade $O(n)$.

Apesar dessa concordância com a teoria, os testes também mostraram que fatores como alocação de memória, cache do processador, implementação do compilador e organização física dos dados podem provocar pequenas diferenças entre o comportamento teórico e o observado na prática.

Essas diferenças reforçam a importância dos benchmarks, pois permitem avaliar o desempenho real das estruturas em situações semelhantes às encontradas em aplicações reais.

Considerações sobre os Resultados:

Os resultados obtidos demonstram que cada estrutura de dados possui características que a tornam mais adequada para determinados tipos de aplicação.

Estruturas como a Tabela Hash são indicadas quando o principal objetivo é realizar consultas rápidas por identificadores. Já a Árvore AVL é mais apropriada para

aplicações que necessitam manter os dados ordenados durante toda a execução. A Skip List apresentou desempenho competitivo com uma implementação relativamente simples, enquanto a Lista Encadeada mostrou-se mais adequada para situações em que predominam inserções e remoções simples.

Dessa forma, a escolha da estrutura deve considerar não apenas sua complexidade teórica, mas também as características do problema, a quantidade de dados processados e as restrições do ambiente onde o sistema será executado.

7. CONCLUSÃO

O desenvolvimento deste projeto permitiu aplicar, na prática, diversos conceitos estudados na disciplina de Estrutura de Dados. Ao longo da implementação, foi possível observar como diferentes estruturas podem influenciar diretamente o desempenho de um sistema, tanto em relação ao tempo de execução quanto ao consumo de memória.

O sistema desenvolvido foi capaz de armazenar, organizar e processar dados provenientes de sensores utilizando cinco estruturas de dados distintas. Além das operações básicas de inserção, busca e remoção, também foi implementado um módulo de detecção de anomalias baseado em média e desvio padrão, permitindo identificar automaticamente leituras fora do comportamento esperado.

Os testes realizados mostraram que cada estrutura possui características próprias que a tornam mais adequada para determinadas situações.

A Tabela Hash apresentou os menores tempos de busca, sendo indicada para aplicações que realizam consultas frequentes por identificadores únicos. A Árvore AVL manteve um desempenho consistente nas operações de busca, inserção e remoção, além de preservar os dados ordenados durante toda a execução. A Skip List apresentou resultados próximos aos da AVL, demonstrando que estruturas probabilísticas podem oferecer um bom desempenho com uma implementação relativamente simples. Já a Lista Encadeada mostrou-se adequada para operações simples de inserção, porém apresentou limitações em buscas realizadas sobre grandes volumes de dados.

Outro ponto importante do trabalho foi o desenvolvimento da versão otimizada da Árvore AVL utilizando bitfields. Embora essa otimização não tenha reduzido significativamente o tempo das operações, ela permitiu diminuir o consumo de memória da estrutura, mostrando que pequenas alterações na organização dos dados podem trazer benefícios importantes, principalmente em sistemas embarcados e dispositivos com recursos limitados.

Os experimentos realizados sob diferentes restrições também evidenciaram que o desempenho das estruturas depende não apenas de sua complexidade teórica, mas

também das condições de execução do sistema. Limitações de memória, processamento, latência e qualidade dos dados influenciaram diretamente os resultados obtidos, reforçando a importância de validar algoritmos em situações próximas às encontradas em aplicações reais.

De forma geral, os resultados confirmaram os conceitos apresentados durante a disciplina e demonstraram que não existe uma única estrutura de dados capaz de atender da melhor forma todos os tipos de problema. A escolha da estrutura mais adequada deve considerar as características da aplicação, o volume de dados processados e as operações realizadas com maior frequência.

Como possibilidade de continuidade deste projeto, podem ser implementadas novas estruturas de dados, como Bloom Filter, KD-Tree ou Segment Tree, ampliando as comparações realizadas neste trabalho. Também seria interessante desenvolver uma interface gráfica para facilitar a visualização dos dados e dos benchmarks, além da utilização de conjuntos de dados maiores e mais variados para avaliar o comportamento das estruturas em cenários ainda mais próximos de aplicações reais.

Em resumo, o projeto atingiu os objetivos propostos, permitindo compreender de forma prática as vantagens, limitações e aplicações das diferentes estruturas de dados estudadas, além de demonstrar a importância da análise experimental na escolha da solução mais adequada para cada contexto.

8. REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DROZDEK, Adam. Estruturas de Dados e Algoritmos em C++. 4. ed. São Paulo: Cengage Learning, 2013.

UCI MACHINE LEARNING REPOSITORY. Human Activity Recognition Using Smartphones Dataset. Disponível em: <https://archive.ics.uci.edu/ml/datasets/human+activity+recognition+using+smartphones>. Acesso em: jun. 2026.

ZOBEL, J.; DART, P.; RAMAMOCHANARAO, K. Trie hashing. In: Proceedings of the 7th International Conference on Data Engineering. IEEE, 1991.